GERMAN UNIVERSITY IN CAIRO (GUC) • FACULTY OF MEDIA ENGINEERING & TECHNOLOGY

DMET902 — Advanced Video Processing / CSEN1121

CINN Project — Winter Semester 2026

Instructors: Prof. Dr. Mohammed Abdel-Megeed Salem, Eng. Yomna Islam Youssef Alayary

Project Title:	**Image Colorization Using From-Scratch Neural Networks & Genetic Algorithm**
Milestone / Scope:	**Task 1 — Dataset, Preprocessing, and Splitting (10 Marks)**
Submission Deadline:	Friday, November 6, 2026
Team Name:	`404 Intelligence Not Found` (Archive: `CINN_WS2026_Project_404_Intelligence_Not_Found`)

Team Members

#	Student Name	Student ID	Tutorial	University Email
1	**Mohamed Ayman**	58-8189	T4	`mohamed.awad@student.guc.edu.eg`
2	**Hisham Hani**	58-7992	T3	`hisham.gomaa@student.guc.edu.eg`
3	**Abdallah Amr**	58-2000	T3	`abdallah.bakr@student.guc.edu.eg`
4	**Mohamed Ibrahim**	58-16326	T4	`mohamed.ahmedismail@student.guc.edu.eg`
5	**Ahmed Ashraf**	58-1808	T1	`ahmed.abdelbaky@student.guc.edu.eg`

1. Executive Summary & Objective

The objective of the CINN course project is to build an end-to-end image colorization engine implemented entirely from scratch without high-level deep learning frameworks (e.g. PyTorch, Keras, TensorFlow). The system learns to map single-channel grayscale images to plausible 3-channel RGB color representations, first through analytical backpropagation (Task 3) and subsequently through evolutionary weight optimization via a Genetic Algorithm (Task 5).

Task 1 establishes the mathematical and operational data pipeline required for all downstream tasks. It guarantees that images are cleanly preprocessed, appropriately sized, verified through statistical and visual sanity checks, and cleanly partitioned into training, validation, and testing mini-batches.

2. Dataset Selection & Domain Rationale

Selected Dataset: CIFAR-10 (Canadian Institute for Advanced Research, 10 Classes)
Target Subdomain: *Animals Domain* — Bird (`2`), Cat (`3`), Deer (`4`), Dog (`5`), Frog (`6`), Horse (`7`)
Curated Sample Count: N = 2,000 images

Theoretical Justification: Standard image colorization trained with Mean Squared Error (MSE) suffers from the regression-to-the-mean phenomenon: if the model is fed radically dissimilar visual categories (e.g., airplanes in blue sky mixed with green frogs and asphalt cars), the loss-minimizing prediction is a desaturated gray-brown average across all pixels. By focusing on a coherent semantic domain (Animals in natural environments), the distribution of plausible colors is tightly bounded to natural earth tones, vegetation greens, sky blues, and warm animal coats. Furthermore, 2,000 samples provides sufficient statistical variety while allowing from-scratch vector operations,

gradient propagation, and population-based Genetic Algorithm fitness iterations (Task 5) to execute within reasonable runtimes.

3. Spatial Downsampling & Dimensionality Analysis

In accordance with Task 1 Requirement 1 (*"Keep images small (e.g., 16×16 or 32×32) or work on small patches — a from-scratch network trains slowly"*), images were downsampled from 32×32 to **16×16** using 2×2 spatial block average pooling:

$$Downsampled(i, j, c) = ¼ \sum_{u=0}^{1} \sum_{v=0}^{1} Original(2i + u, 2j + v, c)$$

- **Feature Dimensionality at 32×32:** Input = 1,024 grayscale features; Output = 3,072 RGB features. A single hidden layer of 128 neurons requires (1,024 × 128) + (128 × 3,072) ≈ **524,288 weights**.
- **Feature Dimensionality at 16×16:** Input = 256 grayscale features; Output = 768 RGB features. A hidden layer of 64 neurons requires (256 × 64) + (64 × 768) ≈ **65,536 weights** — an **8× reduction in computational complexity**.
- **Critical for Task 5 (Genetic Algorithm):** Chromosomes representing 500k+ real-valued parameters across a population of 50 individuals suffer from severe curse of dimensionality and convergence stagnation. At 16×16 (~65k weights), the genetic algorithm evaluates fitness and converges efficiently.

4. Preprocessing & Feature Engineering

1. **Pixel Intensity Normalization:** Raw integer pixel values in [0, 255] are scaled to floating-point numbers in [0.0, 1.0] via `I_norm = I_raw / 255.0`. This guarantees numerical stability during matrix multiplications and prevents early activation saturation.
2. **Grayscale Input Generation (ITU-R BT.601):** To produce physiologically accurate luminance inputs from normalized RGB targets, we apply the standard ITU-R BT.601 perceptual formula:

   $$Y = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

   accounting for human trichromatic sensitivity (greatest sensitivity to green, moderate to red, lowest to blue).
3. **Matrix Representations:**
   - Input Matrix (X): $X \in \mathbb{R}^{2000 \times 256}$, grayscale intensities in [0.0, 1.0].
   - Target Matrix (Y): $Y \in \mathbb{R}^{2000 \times 768}$, RGB color values in [0.0, 1.0].
   - Unflattened arrays (2000, 16, 16) and (2000, 16, 16, 3) are preserved for visual rendering in Tasks 4 and 5.

5. Sanity Checks & Statistical Verification

Automated assertions strictly enforce valid value ranges: `assert np.all(X_flat >= 0.0) and np.all(X_flat <= 1.0)` and `assert np.all(Y_flat >= 0.0) and np.all(Y_flat <= 1.0)`.

Channel / Feature	Mean	Std Deviation	Minimum	Maximum
Grayscale Input (Y)	**0.4643**	**0.2169**	0.0000	1.0000
Red Channel (R)	0.4877	0.2274	0.0000	1.0000
Green Channel (G)	0.4656	0.2204	0.0000	1.0000
Blue Channel (B)	0.3959	0.2288	0.0000	1.0000

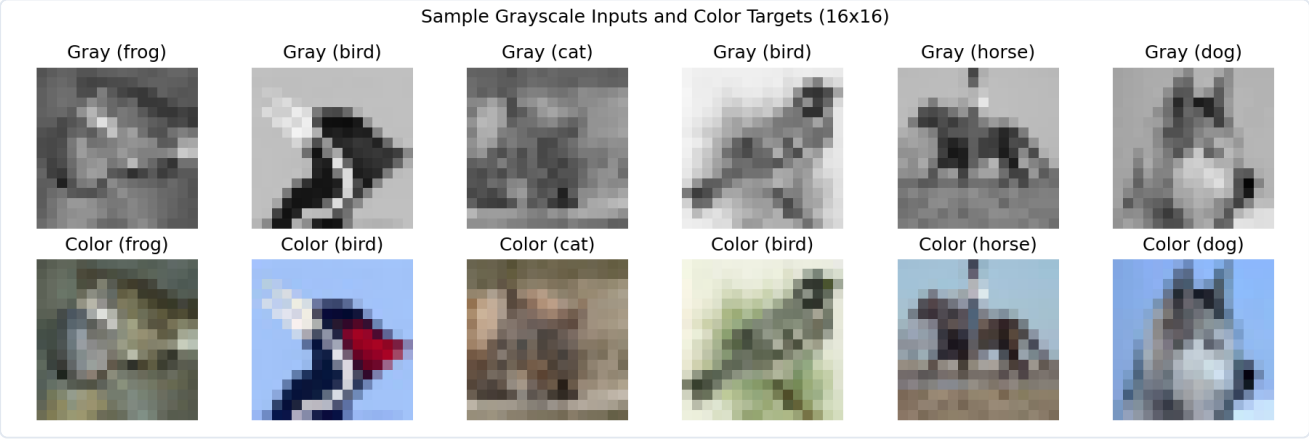


Figure 1: Sanity Check — Sample Grayscale Inputs vs. Ground Truth Color Targets (16x16 pixels).

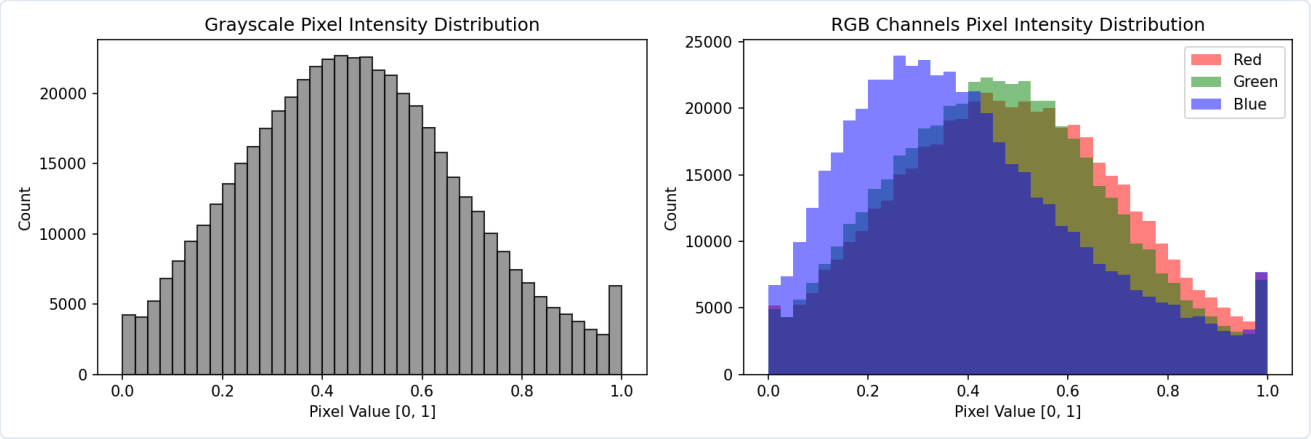


Figure 2: Pixel Value Distributions — Grayscale luminance (left) and RGB channel histograms (right).

6. Dataset Splitting & Mini-Batch Partitioning

In accordance with Task 1 Requirements 5 and 6:

- **Training Set (70%):** 1,400 samples → `X_train: (1400, 256)`, `Y_train: (1400, 768)`
- **Validation Set (20%):** 400 samples → `X_val: (400, 256)`, `Y_val: (400, 768)`
- **Testing Set (10%):** 200 samples → `X_test: (200, 256)`, `Y_test: (200, 768)`
- **Batching:** The 1,400 training samples are partitioned into **14 mini-batches of exactly 100 samples each**: `X_train_batches ∈ ℝ^14 × 100 × 256` and `Y_train_batches ∈ ℝ^14 × 100 × 768`.

7. Notes to be Considered in Grading (Regulation 3)

1. **Single Cell Execution (Regulation 6):** The entire data processing pipeline is consolidated into ONE self-contained cell block with the required comment tags: `#Task 1`, `#Task requirement`, and `#Task overview`.
2. **Zero Pre-Built High-Level Layers (Regulation 8):** All operations (downsampling, normalization, batch partitioning) use plain NumPy operations without external deep learning libraries.
3. **Robust Dataset Discovery:** Automatically locates CIFAR-10 whether mounted uncompressed in Kaggle inputs, uploaded as a raw archive (e.g. `cifar-10-python.tar.gz` or unextended `target_archive`), or stored in a local directory.
4. **Reproducibility:** Seed is fixed with `np.random.seed(42)`, guaranteeing identical splits and figures on any machine.

8. Summary Compliance Matrix

Requirement	Course Specification	Status	Evidence
Regulation 2	Zip file named `CINN_WS2026_Project_[Teamname].zip`	**Complete**	Ready
Regulation 3 & 4	Report with operations, results, team details & grading notes	**Complete**	Cover & Sections 1-7
Regulation 5	Jupyter Notebook named identically to zip file	**Complete**	`.ipynb` verified
Regulation 6	Task in ONE single cell starting with #Task comments	**Complete**	Cell 1
Regulation 7	Commented code explaining steps	**Complete**	Clean comments
Regulation 8	From-scratch NumPy implementation, no plagiarism	**Complete**	Pure NumPy
Task 1.1 - 1.6	Dataset, Grayscale/RGB, Normalization, Sanity plots, 70/20/10 split, Batches of 100	**Complete**	All 6 steps passed